



PROGRAMAÇÃO



Escola Naval



António Gonçalves

AGENDA

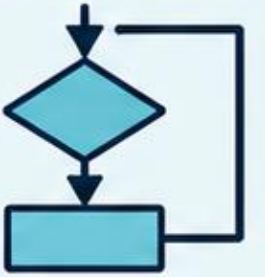
1 Repetição e iteração

Resolver problemas através da repetição de instruções.



2 Funcionamento do while

Compreender a lógica de repetição com while.



3 Contadores e decisões

Utilizar contadores e instruções de decisão em problemas de repetição.



4 Validação e sentinelas

Garantir a entrada de dados válidos e utilizar valores sentinela para terminar repetições.



5 Totais, médias e testes

Calcular totais e médias e realizar testes com repetição.



REPETIÇÃO E ITERAÇÃO

Repetição: executar várias vezes as mesmas instruções.

Três instruções repetidas

```
print("Olá")  
print("Olá")  
print("Olá")
```

Saída

Olá
Olá
Olá

Uma **iteração** é uma execução do corpo do ciclo.

Ciclo while

```
n = 1  
while n <= 3:  
    print("Olá")  
    n = n + 1
```

Saída

Olá
Olá
Olá

Como funciona?

- 1 Iniciar a variável de controle com $n = 1$.
- 2 Enquanto a condição $n \leq 3$ for verdadeira, executar o corpo do ciclo.
- 3 Em cada iteração, imprimir "Olá".
- 4 Depois, aumentar n em 1.
- 5 O ciclo termina quando n for 4, ou seja, quando $n \leq 3$ deixar de ser verdadeira.

O corpo do ciclo é executado três vezes, produzindo três linhas de "Olá".

Do if ao while

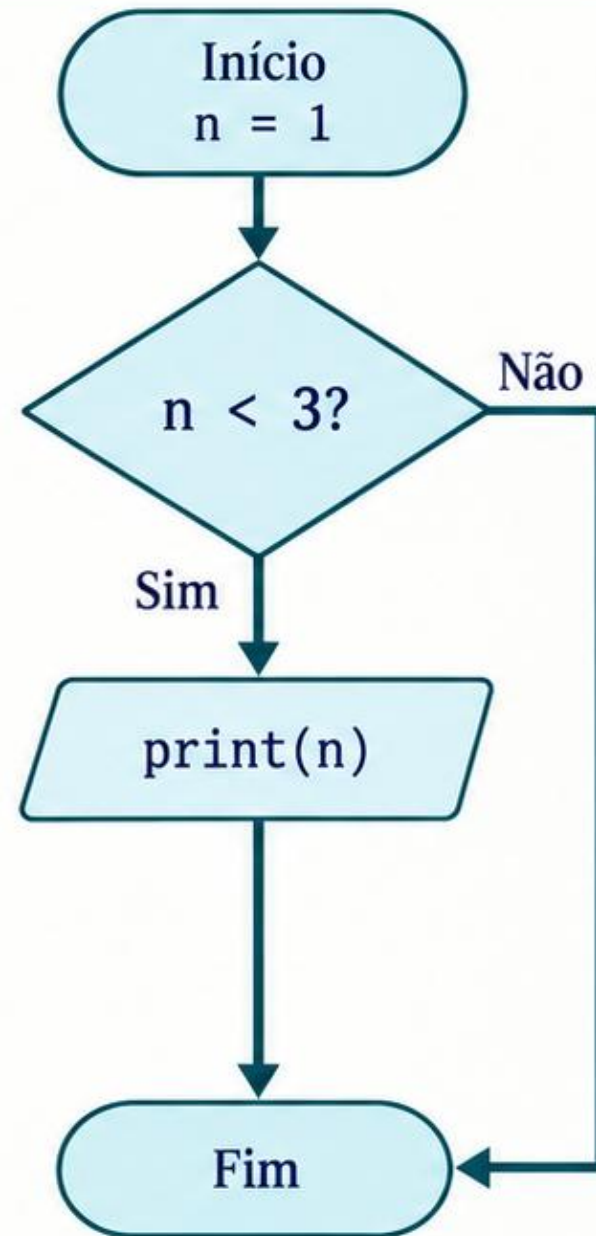
A instrução **if** é avaliada uma única vez.

```
n = 1  
if n < 3:  
    print(n)
```

A condição **n < 3** é testada uma vez. Se for verdadeira, executa o bloco indentado uma vez e termina.

Saída:

1



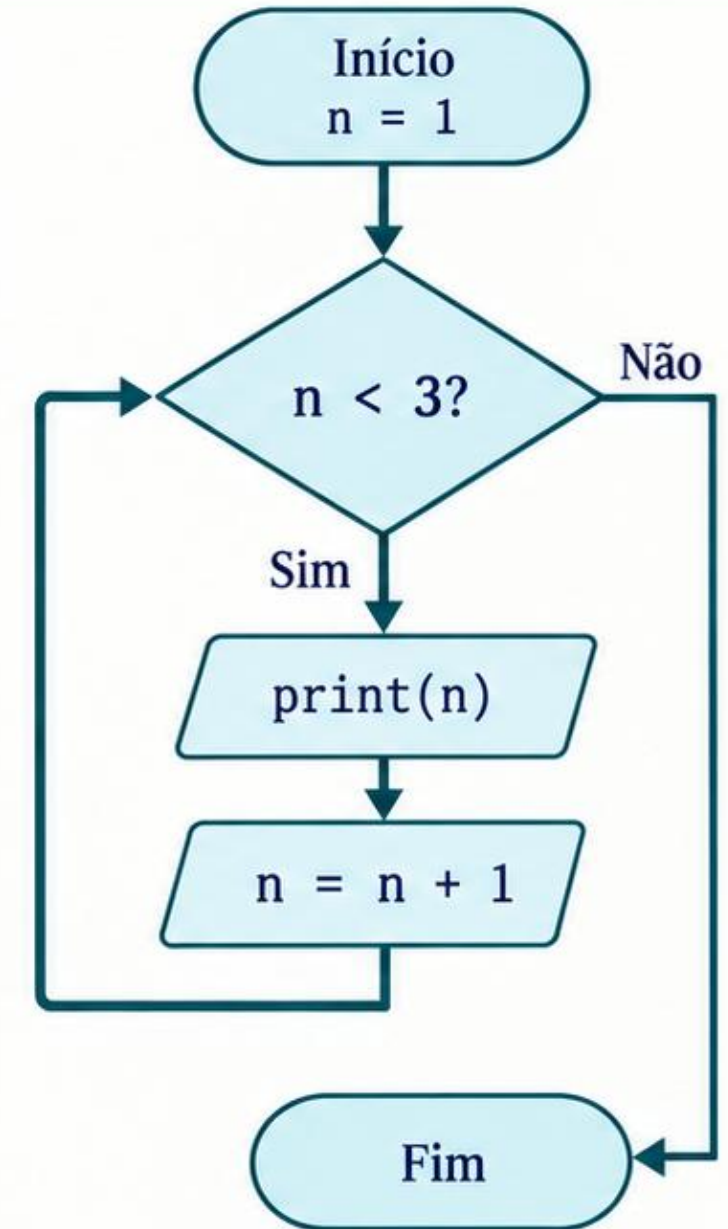
A instrução **while** testa a condição repetidamente.

```
n = 1  
while n < 3:  
    print(n)  
    n = n + 1
```

A condição **n < 3** é testada em cada iteração. Se for verdadeira, executa o bloco e volta a testar. Quando passar a ser falsa, o ciclo termina.

Saída:

1
2



ANATOMIA DO while

Inicialização:

define o valor inicial da variável de controlo.

```
n = 1
```

Condição:

expressão lógica a verificar.

```
while n <= 3:
```

Dois pontos:

indicam o início do bloco.

```
    print(n)
```

```
    n = n + 1
```

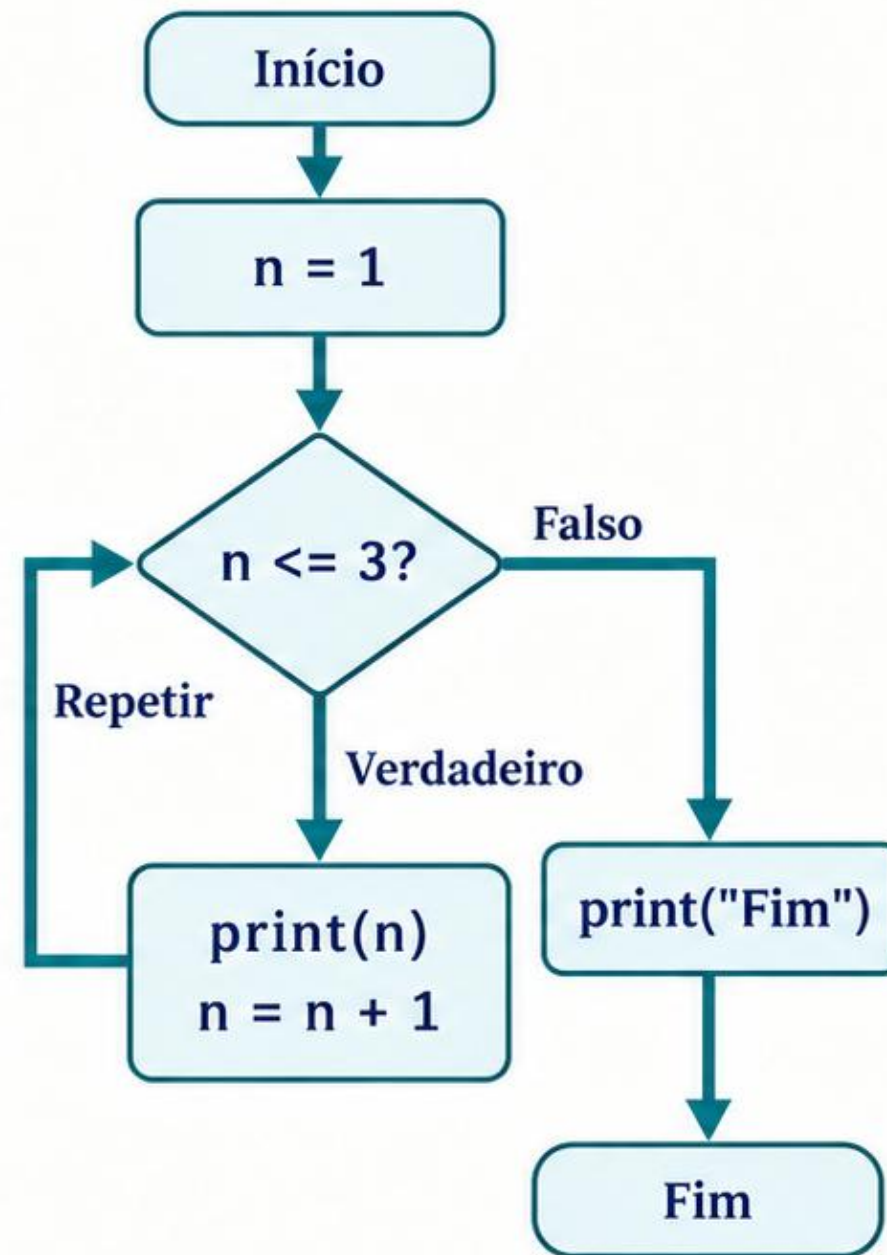
Corpo do ciclo:

instruções executadas enquanto a condição for verdadeira.

```
    print("Fim")
```

Actualização:

modifica a variável de controlo.



1

Inicialização

A variável n é iniciada com 1.

2

Teste da condição

Verifica-se se $n \leq 3$.

3

Execução do corpo

Se a condição for verdadeira, executam-se as instruções indentadas.

4

Actualização

A variável n é incrementada ($n = n + 1$).

5

Continuação

Volta-se ao teste da condição.

6

Saída

Quando a condição é falsa, executa-se a instrução seguinte ao ciclo.

O ciclo **while** repete o seu corpo enquanto a condição for verdadeira.

O teste acontece primeiro

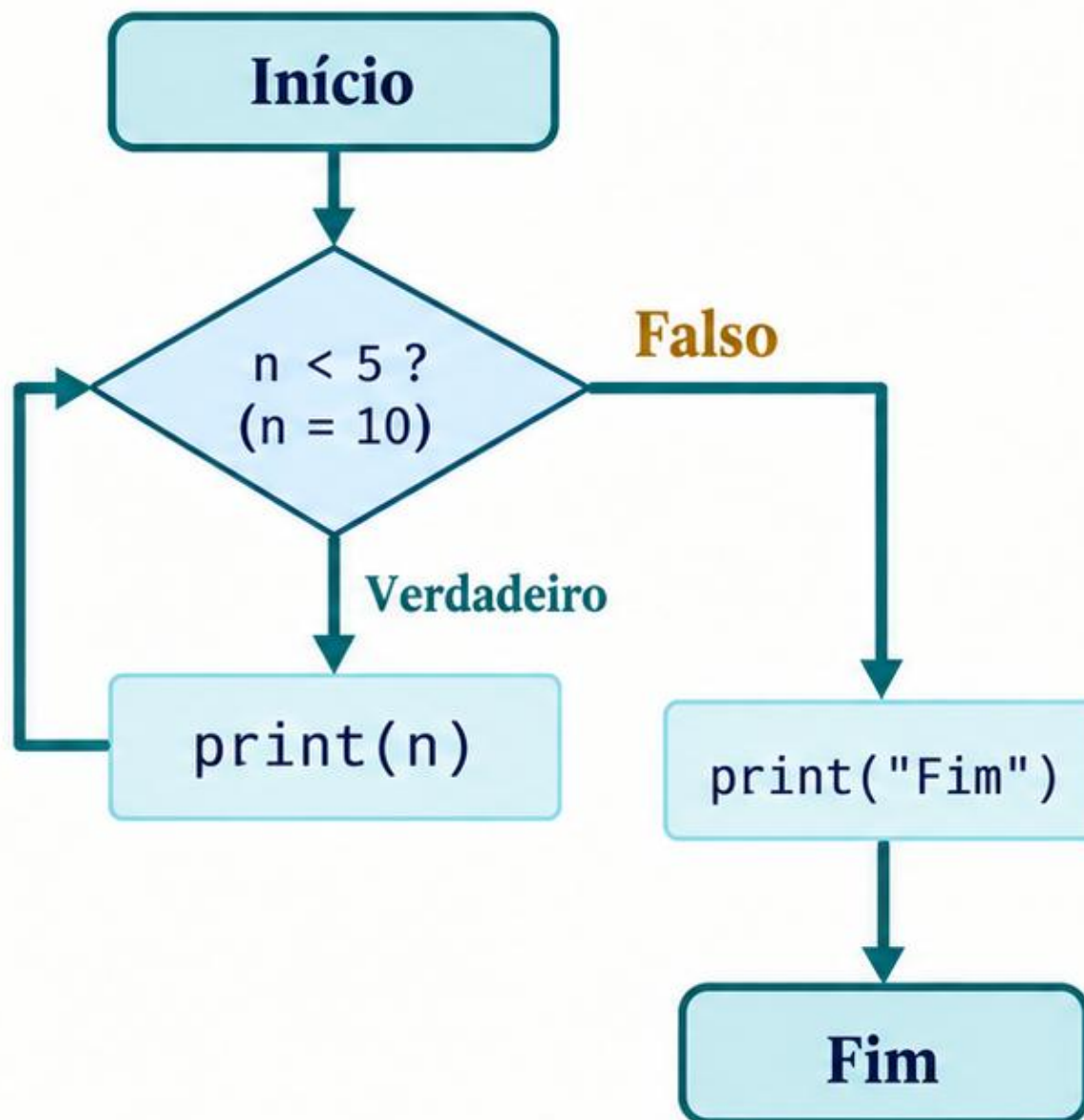
O ciclo **while** testa a condição antes de executar o corpo.

Código Python

```
n = 10
while n < 5:
    print(n)
print("Fim")
```

Saída

Fim



1

Inicialização:

$n = 10$.

2

Teste da condição:

Avalia-se $n < 5$.
Como $10 < 5$ é **Falso**,
o corpo não é executado.

3

Execução após o ciclo:

A instrução `print("Fim")`
é executada.

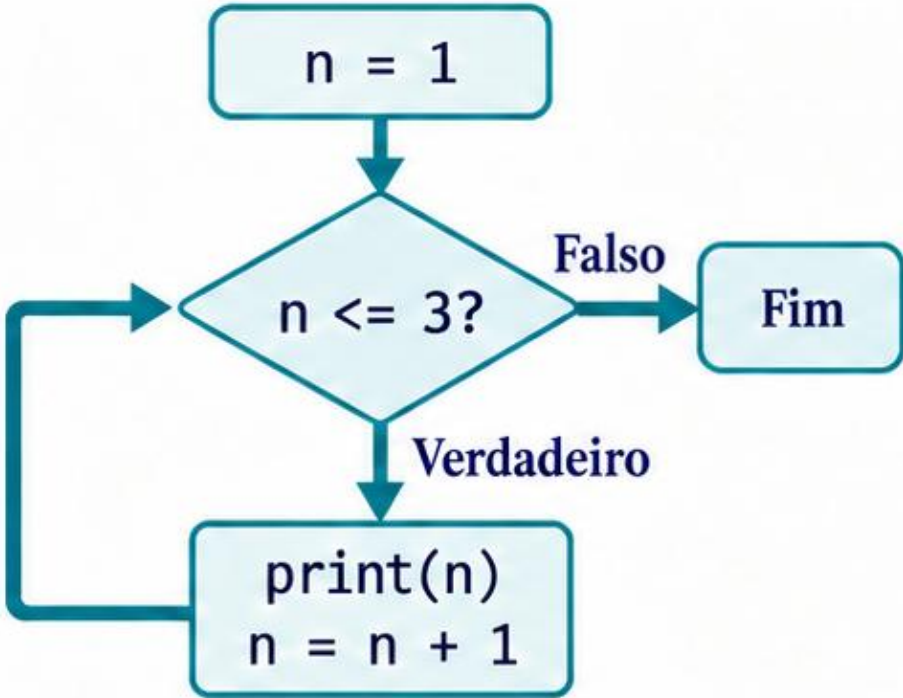
O corpo pode executar zero, uma ou várias vezes.

RASTREAR UM CICLO

Podemos seguir a execução de um ciclo, observando o valor das variáveis em cada iteração.

Código em Python

```
n = 1
while n <= 3:
    print(n)
    n = n + 1
```



n antes do teste	n <= 3	Saída	n após o passo
1	True	1	2
2	True	2	3
3	True	3	4
4	False	—	4

Saída produzida:

1

2

3

Número de iterações:

3

Número de testes da condição:

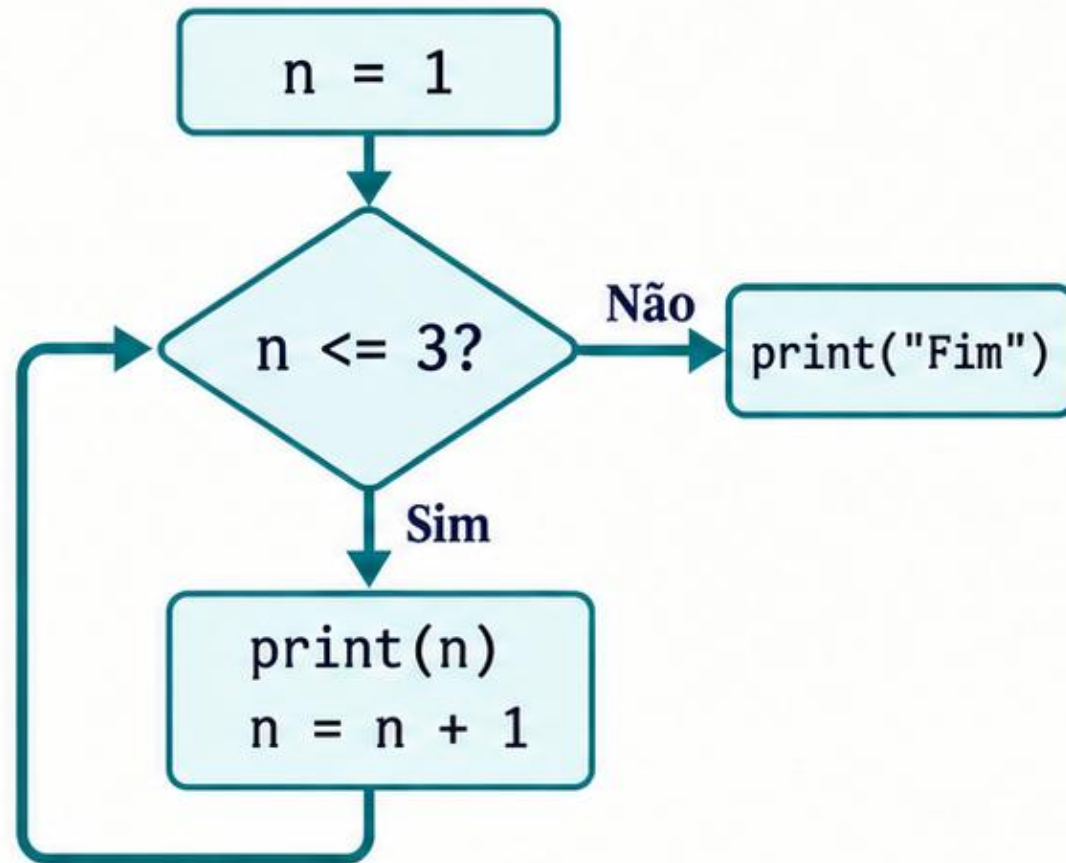
4

Valor final de n:

4

INDENTAÇÃO E ATUALIZAÇÃO

A **indentação** define o bloco de instruções que é executado em cada iteração do ciclo.



```
n = 1
while n <= 3:
    print(n)
    n = n + 1
print("Fim")
```

Corpo do ciclo
(executado em cada iteração).

Instrução **após** o ciclo
(executada uma vez, no final).

Saída
1
2
3
Fim

A atualização tem de ocorrer no ponto previsto em cada iteração.

CICLOS INFINITOS

Código incorreto:

A variável n não é actualizada, mantendo-se com o valor 1. A condição $n \leq 3$ é sempre verdadeira, produzindo uma sequência infinita de 1.

```
n = 1
while n <= 3:
    print(n)
```

Teste	n	Saída
1	1	1
2	1	1
3	1	1
...	1	...

A condição $n \leq 3$ é sempre verdadeira.

Código correto:

A variável n é actualizada em cada iteração, permitindo que a condição se torne falsa e o ciclo termine.

```
n = 1
while n <= 3:
    print(n)
    n = n + 1
```

Teste	n	Saída
1	1	1
2	2	2
3	3	3
4	4	—

No teste seguinte, $4 \leq 3$ é falsa e o ciclo termina.

Para terminar normalmente, a evolução do estado deve tornar a condição falsa.

CONTADORES

Um **contador** controla o número de repetições de um conjunto de instruções, podendo ser usado para gerar sequências ou contar ocorrências.

Código Python

```
n = 1
while n <= 5:
    print(n)
    n = n + 1
```

1. Inicialização

$n = 1$

$n = 1$

2. Comparação

$n \leq 5$

$n \leq 5?$

Não

Fim

Sim

print(n)

$n = n + 1$

3. Atualização

$n = n + 1$

Saída

1
2
3
4
5

O contador é inicializado, testado em cada iteração e atualizado a cada repetição.

CONTAGENS E PASSOS

Contagem decrescente de 1 em 1

Começa em 5 e vai até 1, diminuindo em 1 a cada repetição.

```
n = 5
while n >= 1:
    print(n)
    n = n - 1
```

Sinal ($\geq$)
Limite (1)

Saída produzida

5
4
3
2
1

Contagem de 5
até 1

Contagem crescente de 2 em 2

Começa em 2 e vai até 10, aumentando de 2 em 2 a cada repetição.

```
n = 2
while n <= 10:
    print(n)
    n = n + 2
```

Sinal ($\leq$)
Limite (10)

Saída produzida

2
4
6
8
10

Contagem de 2
até 10
(de 2 em 2)

Contagem decrescente de 2 em 2

Começa em 10 e vai até 0, diminuindo de 2 em 2 a cada repetição.

```
n = 10
while n >= 0:
    print(n)
    n = n - 2
```

Sinal ($\geq$)
Limite (0)

Saída produzida

10
8
6
4
2
0

Contagem de 10
até 0
(de 2 em 2)

DECISÕES DENTRO DO CICLO

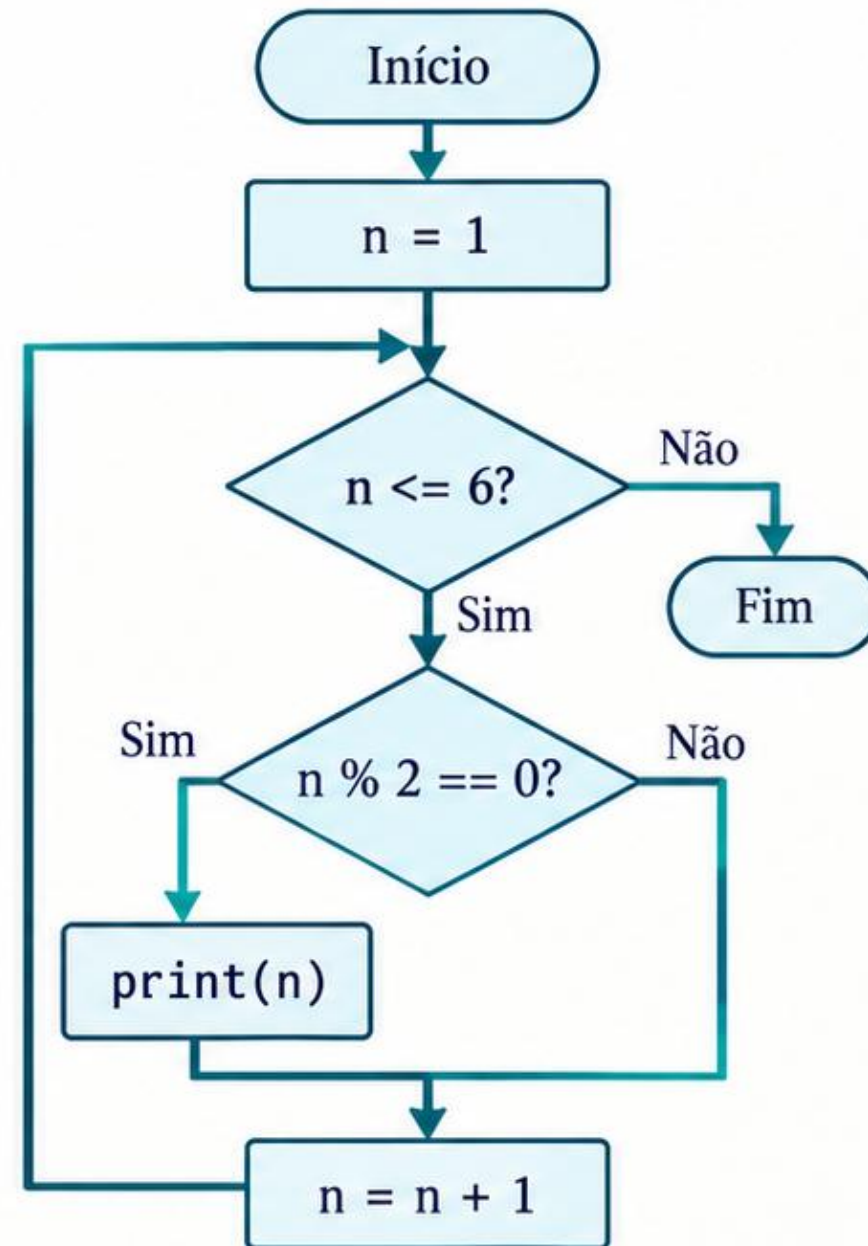
Código em Python

```
n = 1
while n <= 6:
    if n % 2 == 0:
        print(n)
    n = n + 1
```

O **while** percorre os valores.
O **if** seleciona os pares.

Saída do programa

2
4
6



As seis iterações do ciclo

Iteração	Valor de n (no início)	$n \% 2 == 0?$	Imprime?
1	1	Não	Não
2	2	Sim	Sim
3	3	Não	Não
4	4	Sim	Sim
5	5	Não	Não
6	6	Sim	Sim

- O valor de **n** é atualizado em todas as iterações.
- O ciclo percorre exatamente seis valores (1 a 6).
- A instrução **print(n)** é executada apenas quando **n** é par.
- São impressos três valores: 2, 4 e 6.

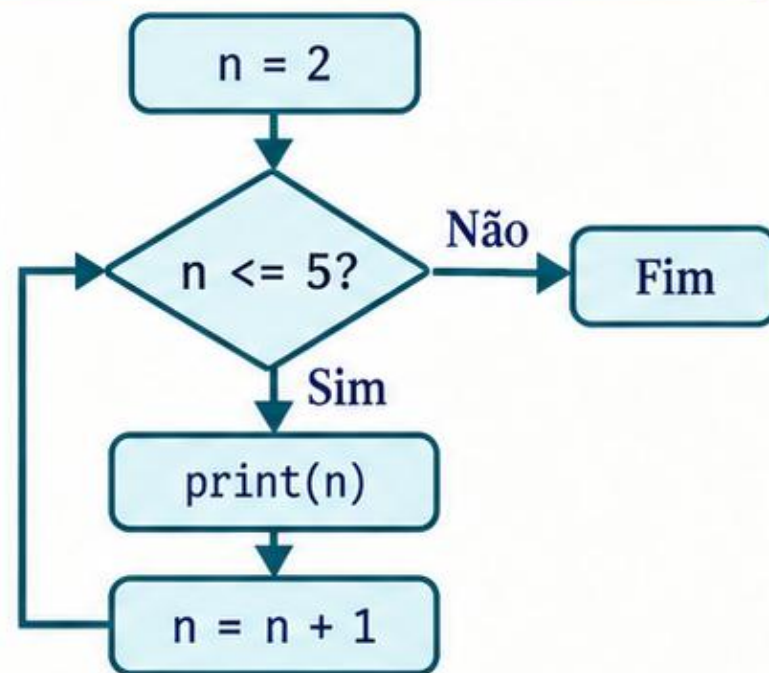
O **while** percorre os valores. O **if** seleciona os pares.

PREVER E CORRIGIR

A

Considere o seguinte excerto de código. Preveja a sua saída, indique o número de iterações do ciclo e o valor final de **n**.

```
n = 2
while n <= 5:
    print(n)
    n = n + 1
```



Resultado

Saída no ecrã:

2
3
4
5

Número de iterações:

4

Valor final de n:

6

B

O seguinte excerto de código pretende contar decrescentemente a partir de 3, mas está incompleto. Corrija o código, preveja a sua saída e indique o valor final de **valor**.

Código incompleto

```
valor = 3
while valor > 0:
    print(valor)
    # falta actualizar valor
```

Código corrigido

```
valor = 3
while valor > 0:
    print(valor)
    valor = valor - 1
```

Resultado (após correção)

Saída no ecrã:

3
2
1

Valor final de valor:

0

VALIDAÇÃO DE ENTRADAS

Repetir até receber um valor válido.

Objetivo: garantir que o utilizador fornece um número positivo.



Código em Python

```
numero = int(input("Número positivo: "))
while numero <= 0:
    numero = int(input("Tente novamente: "))
print("Valor aceite:", numero)
```

Exemplo de execução

```
Número positivo: -5
Tente novamente: 0
Tente novamente: 8
Valor aceite: 8
```

O programa lê um valor, testa se é positivo e, enquanto não for, **volta a pedir um novo valor**.

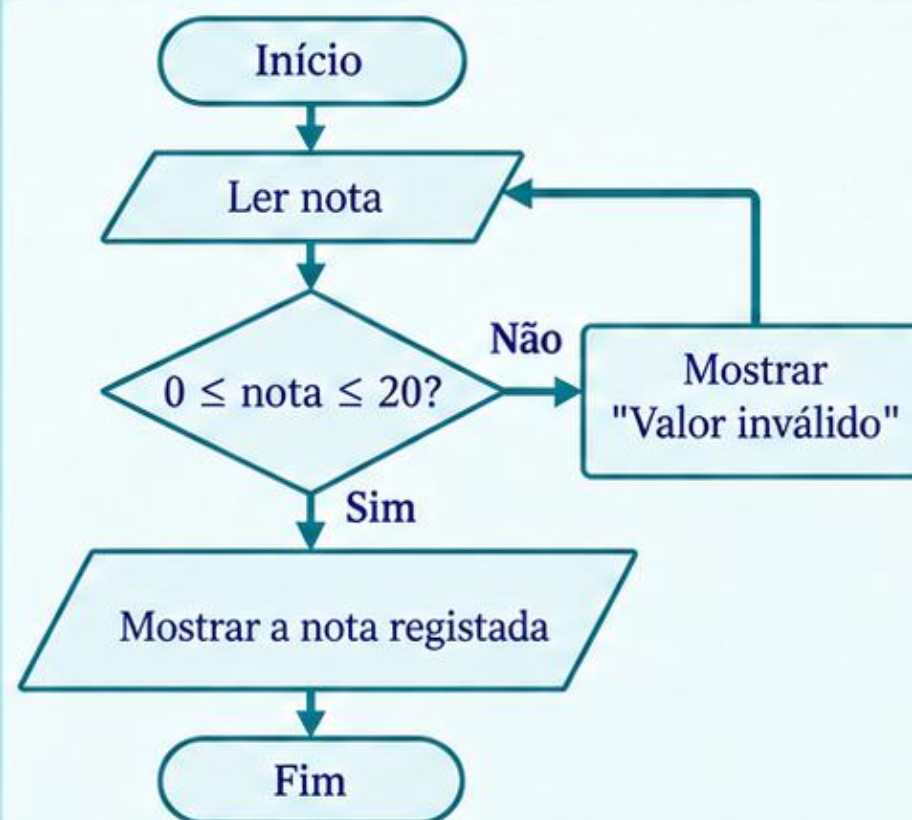
VALIDAR UMA NOTA

Ler uma nota e validar se pertence ao intervalo $[0, 20]$.
Se estiver fora do intervalo, pedir novamente até ser válida.

Código em Python

```
nota=float(input("Nota: "))
while nota<0 or nota>20:
    print("Valor inválido")
    nota=float(input("Nota: "))
print("Nota registada:",nota)
```

Diagrama de fluxo



Exemplo de execução

Nota: 25
Valor inválido
Nota: -3
Valor inválido
Nota: 15
Nota registada: 15.0

1

A nota é válida se estiver no intervalo $[0, 20]$.
Os valores 0 e 20 são válidos.

2

Enquanto a nota estiver fora do intervalo, é mostrado "Valor inválido" e é pedido um novo valor.

3

Quando a nota é válida, é mostrada a mensagem "Nota registada:" seguida do valor lido.

Validar antes de classificar

Código em Python

```
idade = int(input("Idade: "))
while idade < 0:
    idade = int(input("Idade válida: "))

if idade < 18:
    print("Menor de idade")
else:
    print("Maior de idade")
```

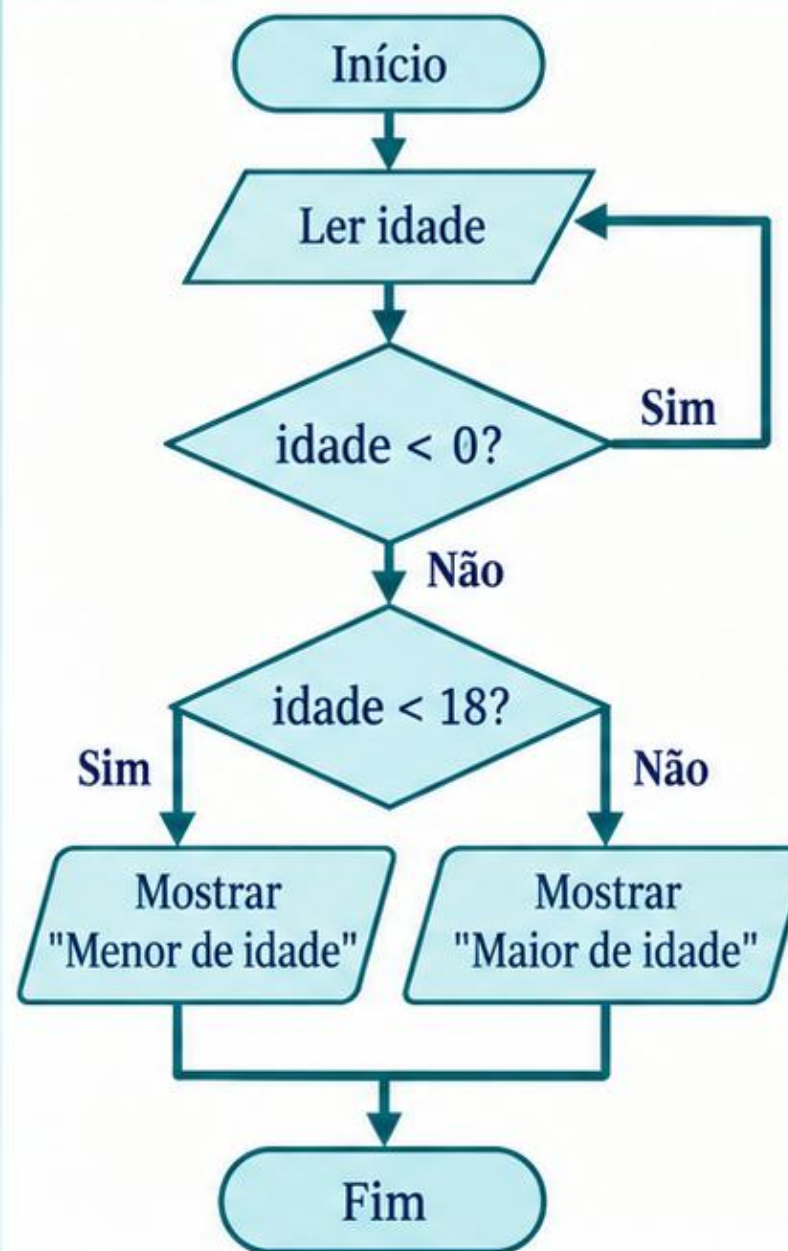
1

Validação: lê a idade e repete a leitura enquanto o valor for negativo.

2

Classificação: após a validação, verifica se a idade é menor que 18.

Fluxograma



Exemplos de execução

Entrada inicial inválida, depois válida

Idade: -2
Idade válida: 17
Menor de idade

Entrada válida à primeira

Idade: 18
Maior de idade

O VALOR SENTINELA

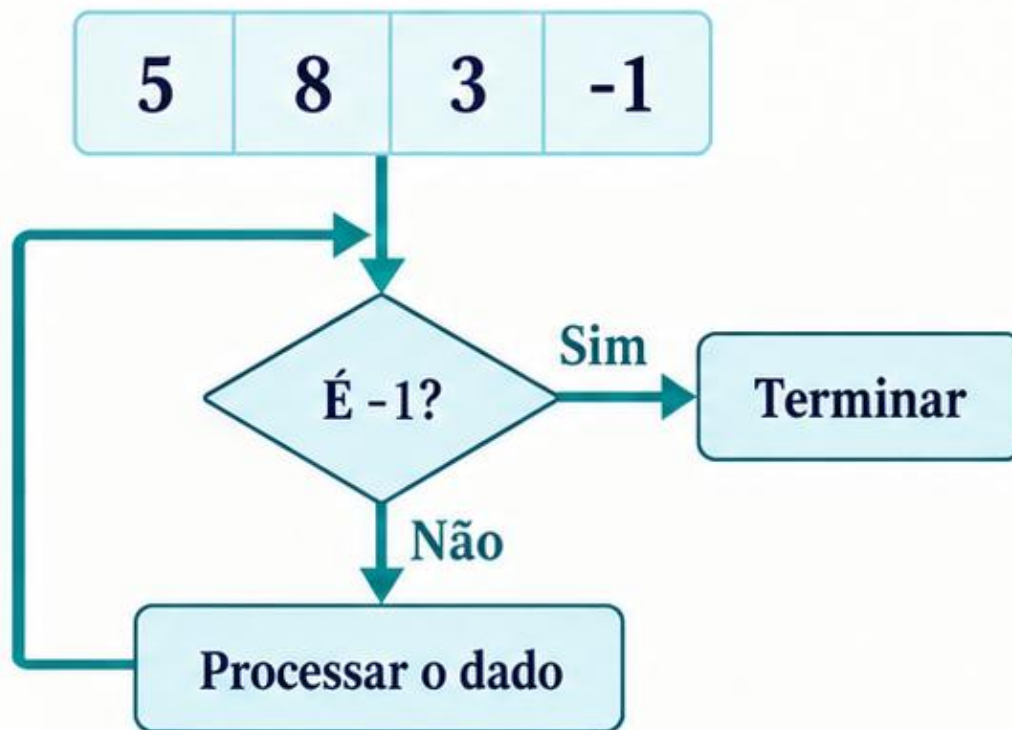
O valor sentinela é um valor especial que indica o fim da sequência de dados. Não é um dado normal.

Dados válidos:
inteiros não negativos
(0, 1, 2, 3, ...).
Sentinela: -1.

Escolha um valor que não se confunda com os dados válidos.

O valor -1 não é adequado, por exemplo, para temperaturas, que podem ser negativas.

Sequência de entrada



Valor lido	Processado?	Observação
5	Sim	
8	Sim	
3	Sim	
-1	Não	A sentinela não é processada.

Exemplo de processamento

Entrada: 5, 8, 3, -1



Dados processados: 5, 8, 3

O algoritmo termina ao ler -1.
A sentinela não é processada.

Escolha um valor que não se confunda com os dados válidos.

LEITURA COM SENTINELA

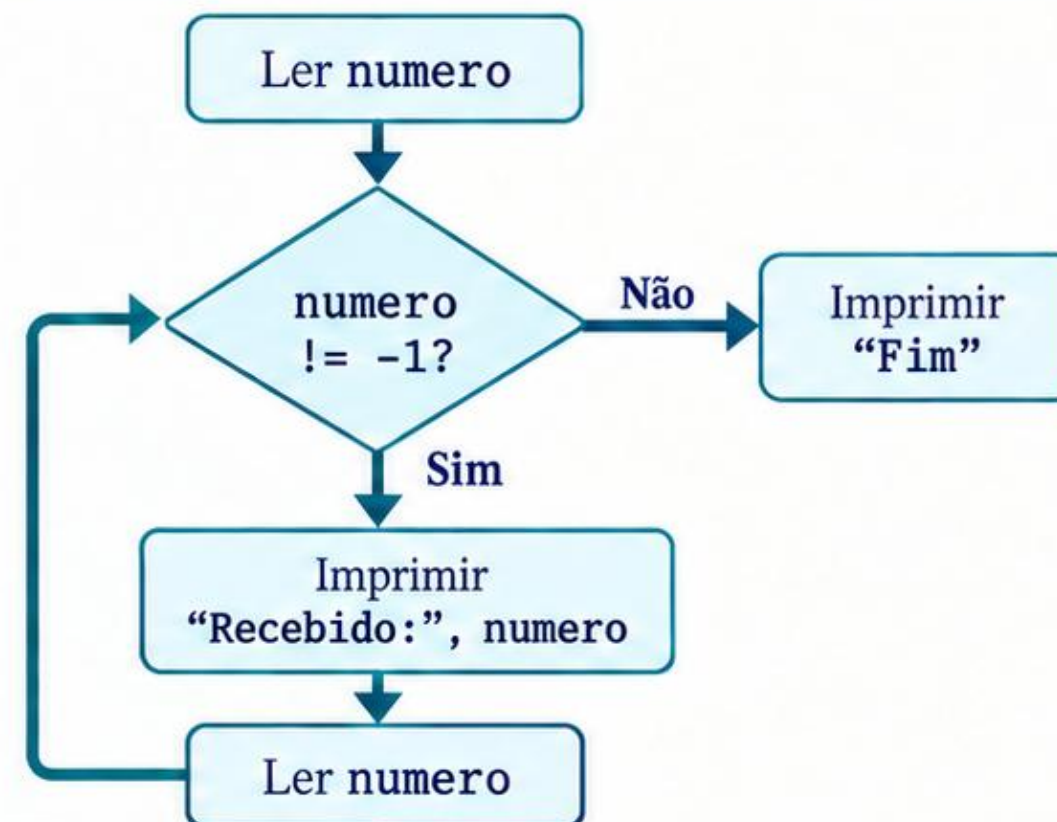
A leitura com sentinela utiliza um valor especial para indicar o fim da introdução de dados, permitindo processar um número indeterminado de valores.

```
numero = int(input("Número (-1 termina): "))  
while numero != -1:  
    print("Recebido:", numero)  
    numero = int(input("Número (-1 termina): "))  
print("Fim")
```

Primeira leitura (antes do ciclo)

Condição de continuação

Nova leitura (dentro do ciclo)



Exemplo 1: entradas 5, 8, -1

Entrada (teclado)

5
8
-1



Saída

Recebido: 5
Recebido: 8
Fim

Exemplo 2: entrada inicial -1

Entrada (teclado)

-1



Saída

Fim

Ideia principal:

enquanto o valor lido for diferente do sentinela (-1), processa-se o valor e lê-se o próximo. Quando for -1, o ciclo termina.

CONTADOR E ACUMULADOR

quantidade = quantidade + 1

conta quantos valores são processados.
Aumenta uma unidade por cada valor.

total = total + valor

soma os valores processados.
Acumula o valor de cada elemento.

Inicialização:

quantidade = 0
total = 0

Por cada valor processado:

quantidade = quantidade + 1
total = total + valor

Valor	Quantidade	Total
5	1	5
10	2	15
3	3	18

Início

quantidade = 0

total = 0

Após 5

quantidade = 1

total = 5

Após 10

quantidade = 2

total = 15

Após 3

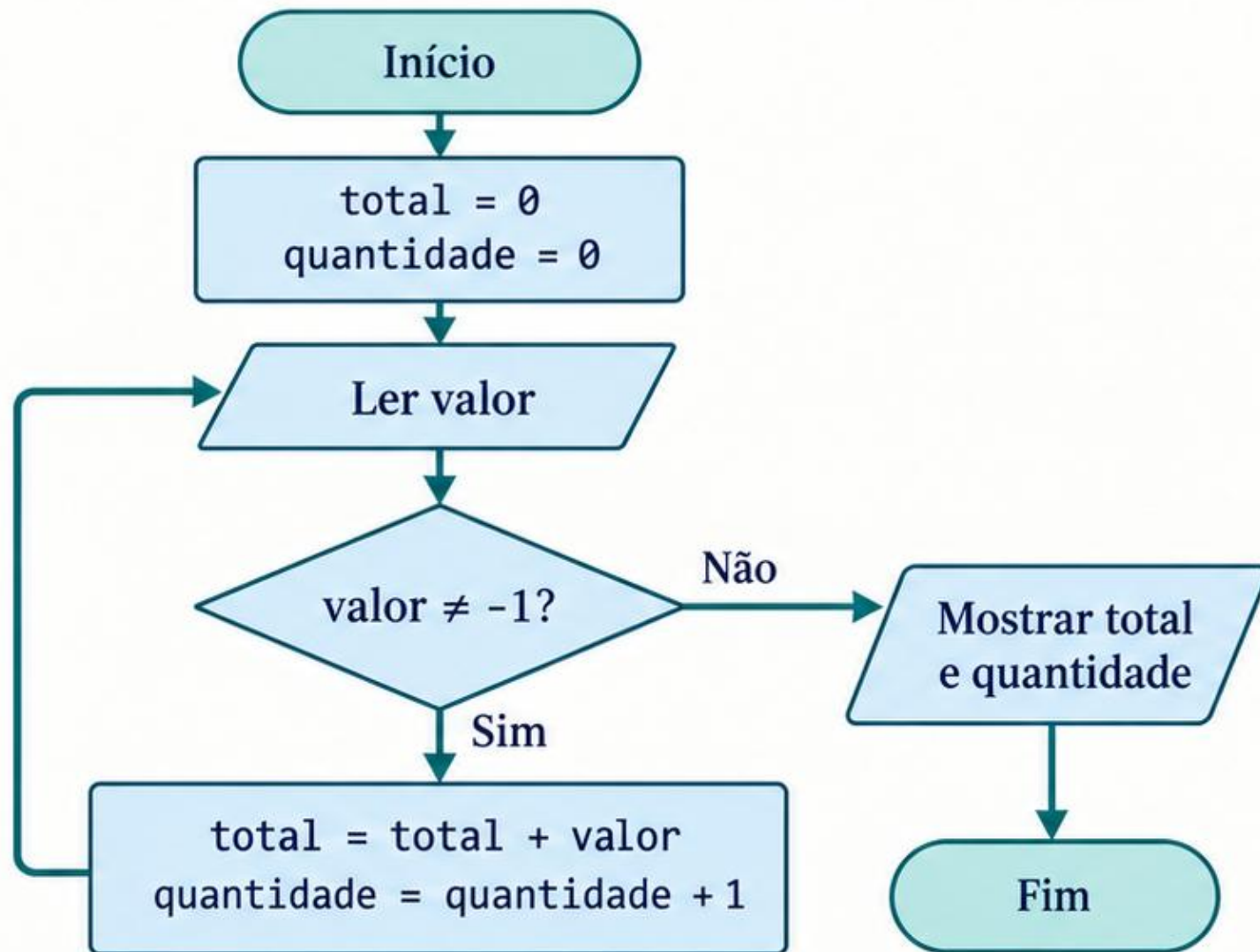
quantidade = 3

total = 18

Contar ocorrências e somar valores respondem a perguntas diferentes.

SENTINELA COM TOTAL E QUANTIDADE

Lê valores inteiros até ser introduzido o valor -1 (sentinela), calculando a soma dos valores e a quantidade de valores lidos. O valor -1 não é incluído nos cálculos.



```
total = 0
quantidade = 0
valor = int(input("Valor (-1 termina): "))
while valor != -1:
    total = total + valor
    quantidade = quantidade + 1
    valor = int(input("Valor (-1 termina): "))
print("Total:", total)
print("Quantidade:", quantidade)
```

Exemplo de execução

Entrada	Saída
Valor (-1 termina): 5	Total: 18 Quantidade: 3
Valor (-1 termina): 10	
Valor (-1 termina): 3	
Valor (-1 termina): -1	



O valor sentinela (-1) termina a leitura e não é incluído no total nem na contagem.

MÉDIA E AUSÊNCIA DE DADOS

Após calcular o total e a quantidade de valores introduzidos, só calculamos a média se existir pelo menos um valor.

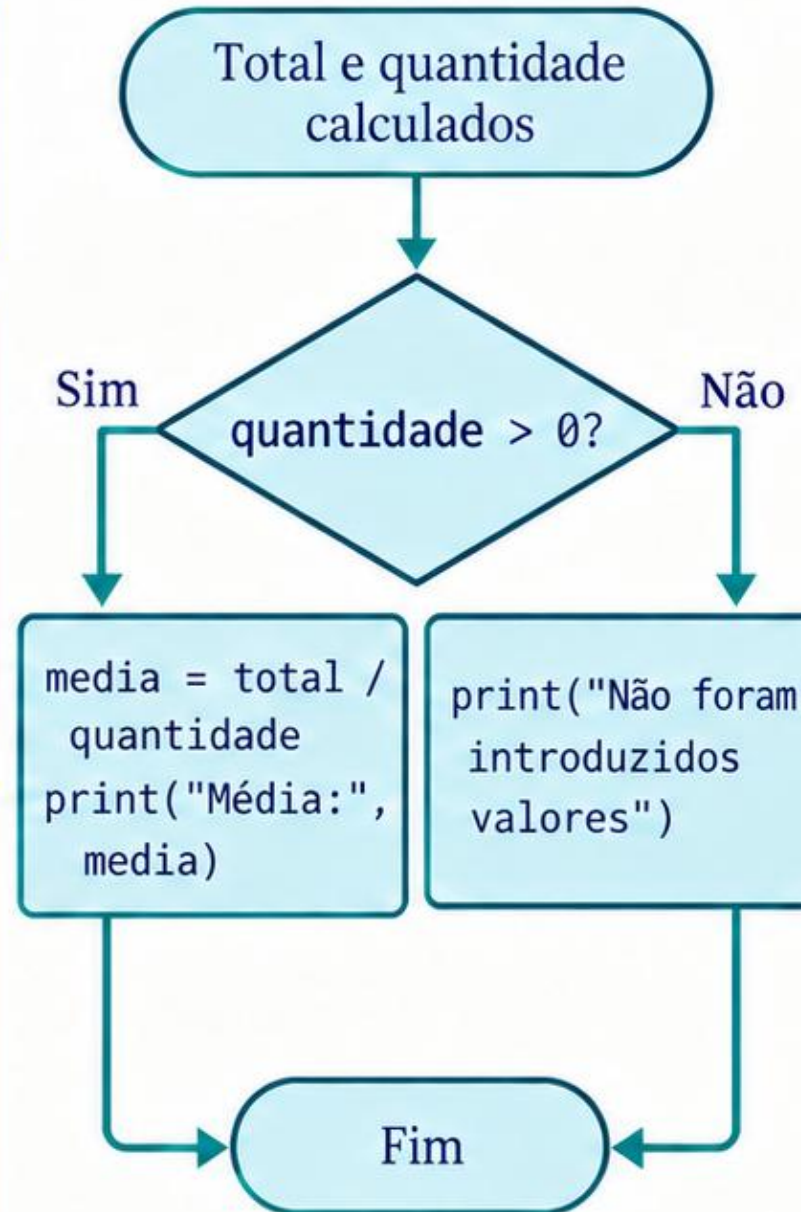
```
if quantidade > 0:
    media = total / quantidade
    print("Média:", media)
else:
    print("Não foram introduzidos valores")
```

Valores iniciais:

Antes de recolher os dados, inicializamos as variáveis com zero.

```
total = 0
quantidade = 0
```

Estes valores indicam que ainda não foram introduzidos dados.



Exemplo de execução

Variável	Valor	Descrição
total	18	Soma dos valores introduzidos
quantidade	3	Número de valores introduzidos
Média	6.0	Resultado de 18 / 3

Exemplo com ausência de dados

Variável	Valor	Descrição
total	0	Valor inicial
quantidade	0	Valor inicial
Saída	Não foram introduzidos valores	Como a quantidade é 0, não se realiza a divisão.

Verifique a quantidade antes de dividir.

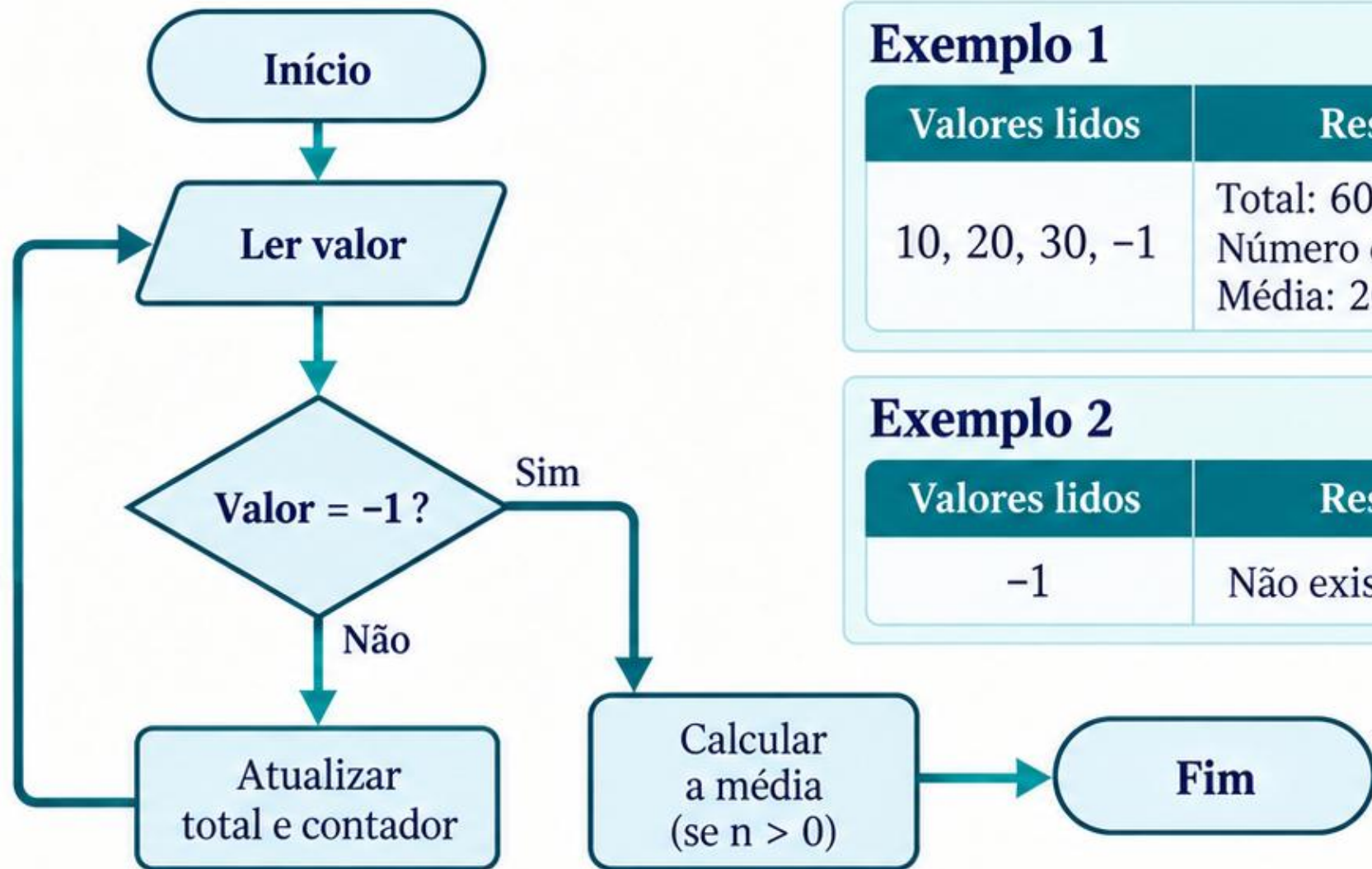
EXERCÍCIO: REGISTO DE VENDAS

Ler valores de vendas (não negativos) até ser introduzido o valor **-1**.
Calcular o total, o número de vendas e a média, apenas se o número de vendas for maior do que zero.

ENTRADA



Valores de vendas
(números não negativos).
O valor **-1** termina a leitura.



Exemplo 1

Valores lidos	Resultado
10, 20, 30, -1	Total: 60 Número de vendas: 3 Média: 20

Exemplo 2

Valores lidos	Resultado
-1	Não existem vendas.

Código (incompleto)

```
total = 0
n = 0
# completar
if n > 0:
    media = total / n
```



Onde ficam a primeira leitura,
a nova leitura e o cálculo da média?

SOLUÇÃO E CASOS DE TESTE

Problema: Ler vendas (números não negativos) até ser introduzido -1. Calcular e apresentar o total das vendas e a média. O valor -1 não deve ser contado.

Solução em Python:

```
total = 0
quantidade = 0
venda = float(input("Venda (-1 termina): "))
while venda != -1:
    total += venda
    quantidade += 1
    venda = float(input("Venda (-1 termina): "))

print("Total:", total)
if quantidade > 0:
    print("Média:", total / quantidade)
else:
    print("Sem vendas")
```

Assunção: Introduzir vendas não negativas ou -1 para terminar.

Casos de teste:

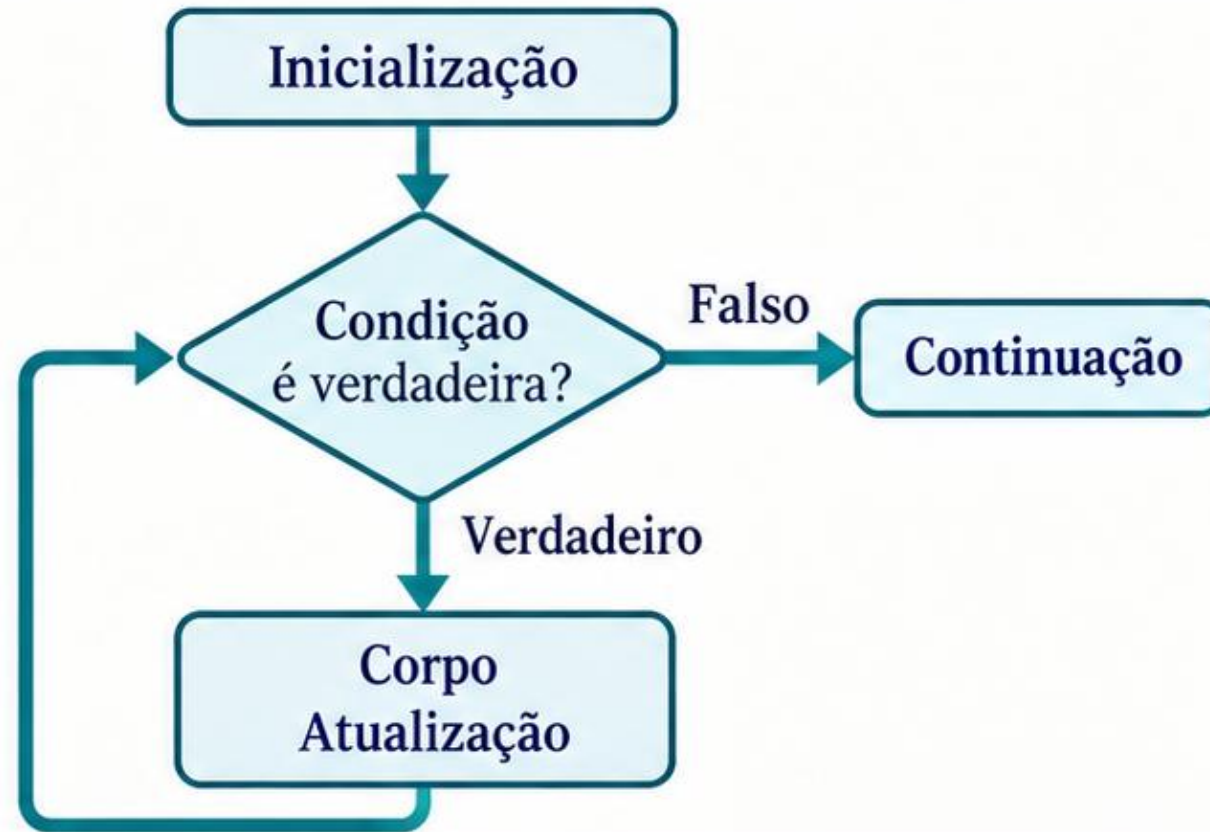
Entradas (introduzidas)	Saída esperada	Explicação
10 20 -1	Total: 30.0 Média: 15.0	Duas vendas (10 e 20). Total 30 e média 15. O valor -1 termina e não é contado.
-1	Total: 0 Sem vendas	Termina de imediato. Não existem vendas.
0 -1	Total: 0.0 Média: 0.0	Uma venda de 0. Total 0 e média 0. O valor -1 termina e não é contado.

O valor -1 termina a leitura e não é incluído no cálculo do total nem da média.

SÍNTESE DO while

Estrutura geral

```
inicialização  
while condição:  
    corpo  
    atualização  
continuação
```



Questões a considerar

- 1 Qual é a condição?
- 2 O que muda em cada iteração?
- 3 Pode executar zero vezes?
- 4 Como termina?

Situações comuns

Validação de entrada



Repete enquanto a entrada for inválida.

Valor sentinela



Termina quando é lido um valor especial que marca o fim.

Contador e acumulador



Um contador contabiliza iterações e um acumulador soma valores.

Teste o caso inicial falso, uma iteração, várias iterações e os valores-limite.